

Laporan Praktikum

“Praktikum Internet of Things (IoT)”

LED, Buzzer, and Push Button

Insyania Cahya Wiguna / 24/533789/SV/23938

Yossi Hasanah

Ardhi Wicaksono Santoso, S.Kom., M.Cs

insyaniacahyawiguna@mail.ugm.ac.id

Teknologi Rekayasa Internet – Departemen Teknik Elektro dan Informatika

Sekolah Vokasi

Universitas Gadjah Mada

Abstrak—*Internet of Things* menjadi salah satu konsep yang penting di dunia digital, karena dapat diimplementasikan di berbagai bidang. Praktikum ini ditujukan untuk memahami prinsip dasar sistem berbasis mikrokontroler melalui pengendalian LED, buzzer dan push button. Kesalahan umum seperti konflik pin, logika button terbalik menjadi masalah yang diangkat. Pada praktikum ini, terdapat 6 *hands on* yang dilakukan, dan hasilnya setiap *hands on* berhasil diimplementasikan.

Kata kunci—*Internet of Things; LED; Buzzer; Push Button*

I. PENDAHULUAN

IoT (Internet of Things) adalah konsep pada jaringan perangkat fisik, peralatan rumah tangga, dan objek fisik lainnya yang dilengkapi dengan sensor, perangkat lunak, dan konektivitas jaringan yang memungkinkan untuk mengumpulkan dan berbagi data. Di era digital sekarang *Internet of Things* memegang peranan penting untuk meningkatkan efisiensi, menghemat biaya dan mengambil keputusan berbasis data [1].

Dalam sistem *Internet of Things*, terdapat perangkat *input* yang digunakan untuk menerima perintah dan mendeteksi kondisi tertentu, dan perangkat *output* digunakan untuk memberikan respon atau notifikasi. Selain itu, terdapat mikrokontroler yang berfungsi sebagai pusat pengendali yang memproses sinyal *input* dan mengatur *output*.

Pada sistem *input* kontrol dasar, perangkat *input* seperti push button digunakan untuk memberikan perintah atau sinyal ke mikrokontroler, sedangkan perangkat *output* seperti LED (*Light Emitting Diode*) dan buzzer berfungsi sebagai indikator visual dan audio. Kesalahan umum pada implementasi sistem sederhana ini masih sering terjadi, terutama pada pemrograman mikrokontroler [3].

Oleh karena itu, praktikum dilakukan untuk memahami prinsip dasar dari control berbasis mikrokontroler untuk dengan memanfaatkan push button sebagai perangkat *input* dan LED (*Light Emitting Diode*) dan buzzer sebagai perangkat *output*. Melalui praktikum ini, pemahaman peserta mengenai logika digital, konfigurasi dan implementasi *IoT (Internet of Things)* sebagai dasar pengembangan sistem yang lebih kompleks.

II. TINJAUAN PUSTAKA

A. Prinsip kerja LED, buzzer, dan push button

LED (*Light Emitting Diode*) adalah komponen elektronika yang dapat memancarkan cahaya monokromatik ketika diberikan tegangan maju. Komponen ini bekerja ketika dialiri bias *forward* dari anoda menuju katoda. Elektron yang berlebih pada material *N-Type* akan berpindah ke wilayah yang bermuatan positif. Saat elektron berada pada wilayah positif, maka *photon* akan dilepaskan dan memancarkan cahaya monokromatik. Spesifikasi LED yang digunakan adalah [2]:

Tabel 1. Spesifikasi LED

Spesifikasi	Ukuran
Tegangan kerja	1,2 – 4,0 V
Arus kerja	20mA
Panjang gelombang	430 – 940 nm

B. Prinsip kerja buzzer

Buzzer adalah sebuah komponen elektronika yang mengubah listrik menjadi getaran suara. Buzzer bekerja dengan tegangan listrik yang diberikan ke bahan *piezoelectric* menyebabkan Gerakan mekanis, kemudian gerakan tersebut diubah menjadi suara yang dapat didengar oleh manusia. Spesifikasi buzzer yang digunakan adalah [2]:

Tabel 2. Spesifikasi buzzer

Spesifikasi	Ukuran
Tegangan kerja	3 - 12 V
Tahanan	16Ohm
Output suara	80 – 85 dB

C. Prinsip kerja push button

Push button adalah sebuah tombol yang menerima *input* saat ditekan oleh pengguna. Cara kerja push button sama dengan saklar lampu dirumah, yaitu memutus dan menyambung listrik. Saat tombol ditekan arus akan menyambungkan arus, dan saat tombol dilepas, aliran arus akan terputus [2].

D. Mikrokontroler sebagai pengendali

Mikrokontroler adalah sistem komputer lengkap yang sudah terintegrasi. Dalam *Internet of Things*, mikrokontroler berperan

sebagai pusat pengendali yang membaca data dari perangkat *input*, memproses informasi tersebut, dan mengirimkan sinyal ke perangkat *output*. Cara kerja mikrokontroler didasarkan pada eksekusi program yang terdiri dari fungsi *setup()* untuk inisialisasi dan fungsi *loop()* yang berjalan terus-menerus untuk menjalankan logika utama [4].

E. Konsep digital input dan output

Dalam sistem digital, sinyal hanya memiliki dua keadaan yang diskrit, yaitu *high* dan *low*. Tegangan *high* direpresentasikan dengan tegangan tinggi, dan *low* sebagai tegangan rendah. Digital *input* adalah sinyal yang diterima mikrokontroler dari perangkat eksternal, dan digital *output* adalah sinyal yang dikirimkan oleh mikrokontroler ke perangkat eksternal [5].

III. METODE PRAKTIKUM

Alat yang digunakan pada praktikum terdiri dari IoT Tools Kit, kabel jumper, kabel USB, PC, power supply, Arduino IDE, NodeMCUESP8266. Bahan yang digunakan adalah *LED*, *buzzer*, *push button*.

Langkah yang dilakukan dimulai dengan instalasi perangkat lunak, instalasi driver com3 dan *library* yang dibutuhkan untuk praktikum. Karena perangkat lunak dan *library* sudah tersedia, Langkah ini tidak dilakukan lagi. Selanjutnya mulai menyiapkan perangkat keras, hubungkan IoT tools kit ke sumber listrik, hubungkan mikrokontroler ke pc untuk melakukan pemrograman.

Setelah *hardware* dan *software* disetup, selanjutnya adalah merangkai alat dan bahan sesuai dengan skema yang dibuat sesuai perintah di *hands on*. Pada *hands on* 1 – 4, pin *LED* dihubungkan ke pin D1 – D8. Untuk *hands on* 5, pin D4 dihubungkan ke *push button*, dan pin *LED* dihubungkan ke pin D0 – D3 dan D5 – D8, *hands on* 6, pin D4 ke *push button* dan pin D3 ke *buzzer*. Pemrograman kode dilakukan sesuai dengan kondisi yang diminta pada *hands on* dan posisi pin yang terhubung.

IV. HASIL DAN ANALISIS

Percobaan yang dilakukan selama praktikum berjumlah enam *hands on*. Gambar dan video hasil praktikum terdapat dalam link berikut : <https://drive.google.com/drive/folders/1HajOX7UM5XNs2NHn1nykbOx3BphB843> dan kode yang digunakan sebagai berikut :

A. Hands on 1

```
1 // Pin LED sesuai GPIO ESP8266 NodeMCU
2 int ledPins[] = {D1, D2, D3, D4, D5, D6, D7, D8};
3
4 void setup() {
5   for (int i = 0; i < 8; i++) {
6     pinMode(ledPins[i], OUTPUT);
7     digitalWrite(ledPins[i], LOW); // pastikan LED mati di awal
8   }
9 }
10
11 void loop() {
12   // Nyalakan 4 LED pertama (index 0-3), matikan 4 LED berikutnya (index 4-7)
13   for (int i = 0; i < 4; i++) {
14     digitalWrite(ledPins[i], HIGH);
15   }
16   for (int i = 4; i < 8; i++) {
17     digitalWrite(ledPins[i], LOW);
18   }
19   delay(1000);
20
21   // Balikkan: matikan 4 LED pertama, nyalakan 4 LED berikutnya
22   for (int i = 0; i < 4; i++) {
23     digitalWrite(ledPins[i], LOW);
24   }
25   for (int i = 4; i < 8; i++) {
26     digitalWrite(ledPins[i], HIGH);
27   }
28   delay(1000);
29 }
```

Gambar 1. Hands On Satu

Gambar 1 adalah kode yang digunakan untuk menyelesaikan perintah pada *hands on* satu, yaitu mengaktifkan 8 LED secara bergantian, ketika 4 LED aktif, 4 LED berikutnya akan mati. Pin yang digunakan untuk mengaktifkan LED adalah D1-D8. Sebelum program dijalankan, semua LED diatur dalam kondisi mati (*LOW*). Selanjutnya saat program dijalankan, empat LED pertama akan diaktifkan, dan empat LED lainnya akan mati, selama waktu yang sudah diatur pada *delay*, yaitu 1000 *millisecond*. Kondisi kedua juga diatur untuk empat LED pertama akan mati, dan empat LED lainnya akan menyala. Waktu yang digunakan (*delay*) masih sama, yaitu 1000 *millisecond* atau satu detik.

Program ini menampilkan perulangan empat LED yang akan menyala setiap satu detik, kemudian empat LED lainnya akan mati pada saat yang sama. Pola ini berlangsung secara berulang, sehingga menghasilkan efek yang menyala secara bergantian. Hasil praktikum yang terdapat pada link menunjukkan bahwa kode berhasil dijalankan dan LED berhasil menyala secara bergantian setiap satu detik.

Keberhasilan ini menunjukkan konfigurasi pin sebagai output digital berjalan dengan benar dan setiap pin mampu menerima sinyal *high* dan *low* sesuai dengan perintah program. Penggunaan fungsi *delay* menjadikan sistem bekerja secara periodik dalam interval yang tetap. Hasil ini menunjukkan kesesuaian antara teori logika digital dan prinsip kontrol sekuensial pada sistem berbasis mikrokontroler.

B. Hands On 2

```
1 // Pin LED NodeMCU
2 int ledPins[] = {D1, D2, D3, D4, D5, D6, D7, D8};
3
4 void setup() {
5   for (int i = 0; i < 8; i++) {
6     pinMode(ledPins[i], OUTPUT);
7     digitalWrite(ledPins[i], LOW); // pastikan LED mati di awal
8   }
9 }
10
11 void loop() {
12   // LED ganjil nyala, genap mati
13   for (int i = 0; i < 8; i++) {
14     if ((i % 2) == 0) { // index 0,2,4,6 = LED ganjil (LED 1,3,5,7)
15       digitalWrite(ledPins[i], HIGH);
16     } else { // index 1,3,5,7 = LED genap (LED 2,4,6,8)
17       digitalWrite(ledPins[i], LOW);
18     }
19   }
20   delay(1000);
21
22   // LED ganjil mati, genap nyala
23   for (int i = 0; i < 8; i++) {
24     if ((i % 2) == 0) {
25       digitalWrite(ledPins[i], LOW);
26     } else {
27       digitalWrite(ledPins[i], HIGH);
28     }
29   }
30   delay(1000);
31 }
```

Gambar 2. Hands On Dua

Gambar 2 adalah kode yang digunakan untuk menyelesaikan perintah pada *hands on* dua, yaitu menyalakan lampu yang berada pada pin ganjil dan genap secara bergantian. Untuk menyimpan nomor pin LED, program menggunakan *array* dan perulangan *for* untuk mengendalikan semua LED secara efisien. Sebelum program dimulai, pastikan dulu lampu untuk mati. Selanjutnya, lampu yang berada pada pin ganjil akan menyala, dan pada saat yang sama, lampu yang berada pada pin genap akan mati. Waktu untuk setiap keadaan ini (*delay*) adalah satu detik. Setelah satu detik, kondisi akan berbalik, lampu pada pin ganjil akan mati, dan pin genap akan menyala. Kondisi akan terus terjadi secara berulang karena menggunakan perulangan (*loop*).

Untuk menentukan pin genap dan ganjil menggunakan rumus penentuan bilangan genap, yaitu apabila hasil bilangan yang dibagi dengan angka dua sama dengan nol, maka itu

adalah bilangan genap. Untuk menerapkan rumus bilangan genap ini, digunakan logika percabangan untuk menghasilkan output yang tepat.

C. Hands On 3

```

1 // Pin LED NodeMCU
2 int ledPins[] = {D1, D2, D3, D4, D5, D6, D7, D8};
3
4 void setup() {
5     for (int i = 0; i < 8; i++) {
6         pinMode(ledPins[i], OUTPUT);
7         digitalWrite(ledPins[i], LOW); // pastikan semua LED mati di awal
8     }
9 }
10
11 void loop() {
12     // Loop dari LED 1 sampai LED 8
13     for (int i = 0; i < 8; i++) {
14         // Nyalakan LED saat ini
15         digitalWrite(ledPins[i], HIGH);
16         // Matikan LED sebelumnya (jika i > 0)
17         if (i > 0) {
18             digitalWrite(ledPins[i - 1], LOW);
19         }
20         delay(200); // jeda antar LED
21     }
22
23     // Matikan LED terakhir (LED 8) sebelum ulang
24     digitalWrite(ledPins[7], LOW);
25 }

```

Gambar 3. Hands On Tiga

Gambar 3 adalah kode yang digunakan untuk menyelesaikan perintah pada *hands on* tiga, yaitu lampu akan menyala satu per satu secara bergantian dari kiri ke kanan. Sama seperti program sebelumnya, untuk menyimpan nomor pin LED, program menggunakan *array* dan perulangan *for* untuk mengendalikan semua LED secara efisien. Pada kondisi awal, seluruh LED diatur dalam keadaan mati. Selanjutnya LED diatur agar saat satu pin LED menyala, maka pin sebelumnya akan mati. Jarak waktu untuk setiap LED adalah 0,2 detik. Pola lampu yang menyala secara bergantian ini memberikan efek seperti lampu berjalan dari kiri kekanan. Setelah LED terakhir menyala, kondisi akan dimulai dari awal lagi, yaitu LED pertama menyala, dan LED terakhir mati.

Pengendalian LED secara berurutan diatur oleh indeks *array*, sehingga tidak perlu menulis untuk setiap pin. Hasil dari percobaan ini menunjukkan LED yang menyala hanya satu untuk satu waktu. Jeda untuk pergantian LED pertama dan selanjutnya yang mengatur kecepatan perpindahan cahaya, sehingga pola yang ditampilkan terlihat jelas.

D. Hands On 4

```

1 // Pin LED NodeMCU
2 int ledPins[] = {D1, D2, D3, D4, D5, D6, D7, D8};
3
4 void setup() {
5     for (int i = 0; i < 8; i++) {
6         pinMode(ledPins[i], OUTPUT);
7         digitalWrite(ledPins[i], LOW); // pastikan semua LED mati
8     }
9 }
10
11 void loop() {
12     // LED maju dari kiri ke kanan
13     for (int i = 0; i < 8; i++) {
14         digitalWrite(ledPins[i], HIGH);
15         if (i > 0) digitalWrite(ledPins[i - 1], LOW); // matikan LED sebelumnya
16         delay(150);
17     }
18
19     // LED mundur dari kanan ke kiri
20     for (int i = 6; i >= 0; i--) {
21         digitalWrite(ledPins[i], HIGH);
22         digitalWrite(ledPins[i + 1], LOW); // matikan LED sebelumnya
23         delay(150);
24     }
25 }

```

Gambar 4. Hands On Empat

Gambar 4 adalah kode yang digunakan untuk menyelesaikan perintah pada *hands on* empat, yaitu lampu akan menyala satu per satu secara bergantian dari kiri ke kanan, kemudian langsung dilanjutkan menuju kanan ke kiri. Kondisi

ini dikenal dengan *knight riders* yang biasa diterapkan pada kendaraan. Untuk menyimpan nomor pin LED, program menggunakan *array* dan perulangan *for* untuk mengendalikan semua LED secara efisien. Pada percobaan ini, LED menyala secara berurutan dari LED 1 – LED 8 secara berurutan. Setiap LED yang menyala akan mematikan LED yang sebelumnya, sehingga hanya satu LED yang aktif dalam satu waktu.

Percobaan ini hampir sama dengan percobaan yang sebelumnya, namun pada percobaan ini, ketika LED ke-8 sudah menyala, akan dilanjutkan oleh LED ke-7, dan LED sebelumnya akan mati. Pola ini menunjukkan efek LED yang menyala bolak balik dan berlangsung secara terus menerus karena dalam fungsi perulangan. Perulangan *for* diterapkan pada kedua arah yang berbeda. Untuk *for* yang pertama terdapat fungsi *if* yang digunakan untuk mencegah akses indeks negatif, dan indeks yang berada pada *for* kedua dimulai dari indeks ke-6, yang bertujuan untuk mencegah LED terakhir menyala dua kali secara berturut-turut.

E. Hands On 5

```

1 void setup() {
2     pinMode(D4, INPUT_PULLUP); // set S1 sebagai input pull-up
3     pinMode(D0, OUTPUT); // set D8 sebagai output LED
4     pinMode(D1, OUTPUT);
5     pinMode(D2, OUTPUT);
6     pinMode(D3, OUTPUT);
7     pinMode(D8, OUTPUT);
8     pinMode(D5, OUTPUT);
9     pinMode(D6, OUTPUT);
10    pinMode(D7, OUTPUT);
11 }
12
13 void loop() {
14    int buttonState = digitalRead(D4); // baca status tombol
15
16    if (buttonState == HIGH) { // jika tombol ditekan
17        digitalWrite(D0, HIGH);
18        digitalWrite(D1, HIGH);
19        digitalWrite(D2, HIGH);
20        digitalWrite(D3, HIGH);
21        digitalWrite(D8, HIGH);
22        digitalWrite(D5, HIGH);
23        digitalWrite(D6, HIGH);
24        digitalWrite(D7, HIGH);
25
26        delay(100); // tunggu 0.5 detik (efek blink)
27    }
28    else
29        if (buttonState == LOW) { // jika tombol tidak ditekan
30            digitalWrite(D0, LOW);
31            digitalWrite(D1, LOW);
32            digitalWrite(D2, LOW);
33            digitalWrite(D3, LOW);
34            digitalWrite(D8, LOW);
35            digitalWrite(D5, LOW);
36            digitalWrite(D6, LOW);
37            digitalWrite(D7, LOW);
38
39            delay(100); // tunggu 0.5 detik
40
41            digitalWrite(D0, HIGH);
42            digitalWrite(D1, HIGH);
43            digitalWrite(D2, HIGH);
44            digitalWrite(D3, HIGH);
45            digitalWrite(D8, HIGH);
46            digitalWrite(D5, HIGH);
47            digitalWrite(D6, HIGH);
48            digitalWrite(D7, HIGH);
49
50            delay(100);
51        }
52    }
53 }

```

Gambar 5. Hands On Lima

Gambar 5 adalah kode yang digunakan untuk menyelesaikan perintah pada *hands on* lima, yaitu lampu akan berkedip setiap push button ditekan, dan ketika push button dilepas lampu akan mati. Berbeda dengan kode percobaan sebelumnya, pada kode ini, pin LED dijelaskan satu persatu. Pin D4 diatur sebagai pin *input* untuk *push button*. Pin untuk LED berperan sebagai pin *output*. Selanjutnya dilakukan

percabangan, apabila *input push button* bernilai *high* (ditekan), maka *output* untuk LED akan *high* (menyala), dan ketika *input push button* bernilai *low*, maka LED tidak akan menyala.

Penggunaan mode *input pullup* pada pin D4 (*push button*) bertujuan untuk menjaga kestabilan logika pada pin *input* dengan memanfaatkan resistor *pull up*, sehingga dapat mencegah *floating* dan meningkatkan keandalan pembacaan sinyal digital. Jeda waktu pada program digunakan untuk mengatur lama waktu LED menyala dan mati, yaitu 100 *millisecond*. Pola ini akan terjadi secara terus sampai *push button* berubah kondisi.

F. Hands On 6

```
1 void setup() {
2
3   pinMode(D4, INPUT_PULLUP);
4   pinMode(D3, OUTPUT);
5 }
6
7
8 void loop() {
9
10  int buttonState = digitalRead(D4);
11
12  if (buttonState == LOW) {
13
14    tone(D3, 1000);
15    delay(200);
16
17    tone(D3, 500);
18    delay(200);
19  }
20  else {
21
22    noTone(D3);
23
24  }
25 }
26 }
```

Gambar 6. Hands On Enam

Gambar 6 adalah kode yang digunakan untuk menyelesaikan perintah pada *hands on* enam, yaitu *buzzer* akan berbunyi setiap *push button* ditekan. Pada percobaan ini, pin yang digunakan hanya dua, satu pin terhubung ke *push button*, dan pin lainnya terhubung ke *buzzer*. *Input* berada pada *push button* dan *output* berada pada *buzzer*. *noTone* berarti tidak ada bunyi (diam).

Ketika *push button* bernilai *low* (ditekan), maka *buzzer* akan berbunyi selama 200 milidetik, dengan nada bergantian dengan frekuensi 1000 Hz dan 500 Hz. Ketika *push button* tidak ditekan, maka *buzzer* akan diam. Jeda waktu disini untuk menentukan seberapa lama frekuensi berbunyi. Pola pergantian

bunyi akan terjadi selama *push button* ditekan, dimana setiap 200 milidetik, frekuensi bunyi akan berubah.

V. KESIMPULAN

Semua percobaan berhasil diimplementasikan dengan baik. Semua pola pengendalian dapat dijalankan berdasarkan logika yang dirancang. Salah satu langkah untuk mempermudah pengelolaan banyak pin adalah menggunakan array. Logika *high* yang mempresentasikan kondisi aktif dan logika *low* yang merepresentasikan kondisi mati dapat berfungsi sesuai teori.

Perubahan nilai *delay* mempengaruhi kecepatan respon sistem karena menentukan berapa lama mikrokontroler berhenti sebelum menjalankan baris program berikutnya. *Pitches.h* digunakan untuk menyimpan daftar frekuensi nada dalam bentuk konstanta sehingga memudahkan pembuatan suara atau melodi menggunakan *buzzer*.

DAFTAR PUSTAKA

- [1] "What is the Internet of Things (IoT)?," IBM, [Online]. Available: <https://www.ibm.com/think/topics/internet-of-things>. [Accessed 24 February 2026].
- [2] Elok, [Online]. Available: <https://elok.ugm.ac.id/course/section.php?id=231973>. [Accessed 25 February 2026].
- [3] Y. Xu and C. Fischione, "Real-Time Scheduling in LTE for Smart Grids," *Proc. 5th Int. Symp. Commun. Control Signal Process.*, no. May, pp. 2--4, 2012.
- [4] Universitas Komputer Indonesia, "Aplikasi Mikrokontroler dalam Sistem Komputer Dari IoT hingga Otomasi," 2025. [Online]. Tersedia: <https://web.unikom.ac.id/aplikasi-mikrokontroler-dalam-sistem-komputer-dari-iot-hingga-otomasi/> [Diakses: 25 Februari 2026].
- [5] BMCC OpenLab, "Digital Input and Output," 2024. [Online]. Tersedia: <https://openlab.bmcc.cuny.edu/mea211/digital-input-and-output/>